第 3 章 链表和 list

1. 链表的概念

（以下内容可以用 ppt 讲解，课件上总结）

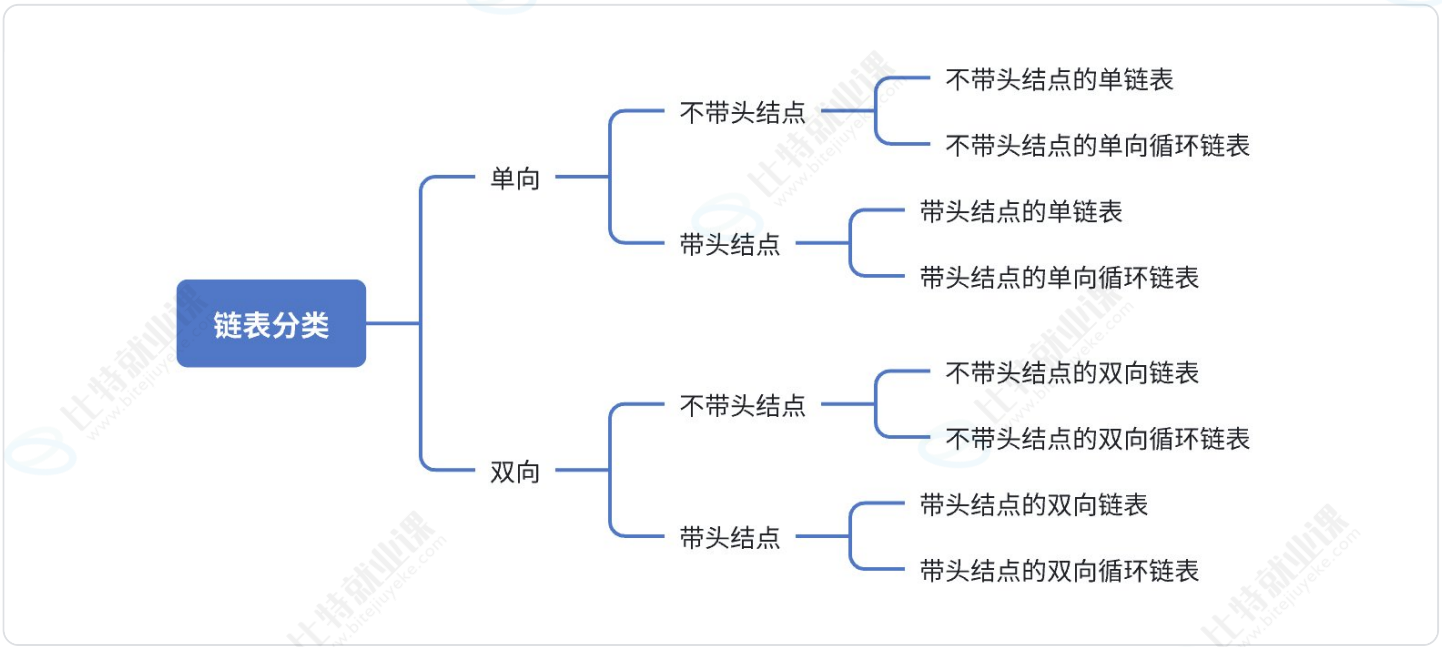
1.1 链表的定义

线性表的**链式存储**就是**链表**。

它是将元素存储在物理上任意的存储单元中，由于无法像顺序表一样通过下标保证数据元素之间的逻辑关系，链式存储除了要保存数据元素外，还需额外维护数据元素之间的逻辑关系，这两部分信息合称**结点**（node）。即结点有两个域：保存数据元素的**数据域**和存储逻辑关系的**指针域**。

1.2 链表的分类

将各种类型的链表排列组合，总共有8种不同链表结构：



虽然链表种类较多，我们只需掌握**单向链表**，**双向链表**和**循环链表**即可，熟悉之后其他链表便可自行实现。

2. 链表的模拟实现

2.1 单链表的模拟实现

2.1.1 实现方式

链表的实现方式分为**动态实现**和**静态实现**两种。

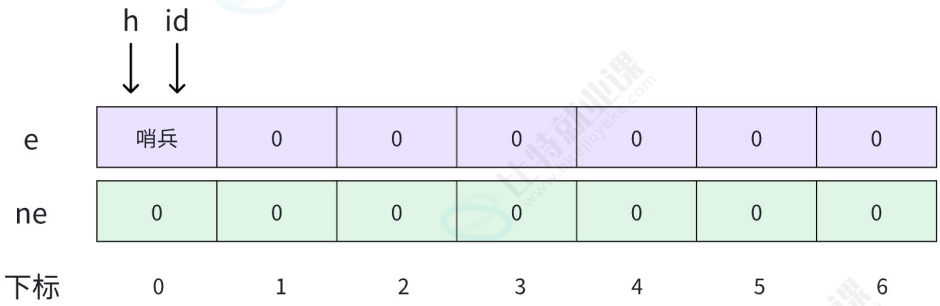
（ppt 演示两种实现方式的区别）

- 动态实现是通过 new 申请结点，然后通过 delete 释放结点的形式构造链表。这种实现方式最能体现链表的特性；
- 静态实现是利用两个数组配合来模拟链表。第一次接触可能比较抽象，但是它的运行速度很快，在算法竞赛中会经常会使用到。

这里建议，如果学有余力，一定要去看看语言课中【加餐】链表。

2.1.2 定义 - 创建 - 初始化

1. 两个足够大的数组，一个用来存数据，一个用来存下一个结点的位置
2. 变量 `h`，充当头指针，表示头结点的位置
3. 变量 `id`，为新来的结点分位置




```cpp
const int N = 1e5 + 10;

int h; // 头指针
int id; // 下一个元素分配的位置
int e[N], ne[N]; // 数据域和指针域

// 下标 0 位置作为哨兵位
// 其中 ne 数组全部初始化为 0，其中 ne[i] = 0 就表示空指针，后续没有结点
// 当然，也可以初始化为 -1 作为空指针，看个人爱好

/*
    e[i] 和 ne[i] 是绑定在一起使用的，也有一种写法是定义一个结构体，把这两个变量放在一起，比如:

    struct node
```

```
    {
        int e, ne;
    }list[N];

    但是，定义成结构体之后，代码书写不方便。我们只要知道 e[i] 和 ne[i] 是绑定在一起使用的即可
*/
```

2.1.3 头插

（用 ppt 演示头插操作的细节）

> 这是链表中最常用也是使用最多的操作，后续树和图的存储中的邻接表以及链式前向星就会用到这个操作。
>
> **因此必须要掌握的操作。**

代码实现：

```cpp
// 头插
void push_front(int x)
{
    // 先把 x 放在一个格子里面
    id++;
    e[id] = x;

    // 修改指针，顺序不能颠倒！
    // 1. x 的右指针指向哨兵位的后继
    // 2. 哨兵位的右指针指向 x
    ne[id] = ne[h];
    ne[h] = id;
}
```

时间复杂度：

> 只涉及指针的修改，时间复杂度为 $O(1)$。

2.1.4 遍历链表

（用 ppt 展示一下遍历链表的流程）

代码实现：

```cpp
// 打印链表
void print()
{
    // 定义一个指针从头结点开始
    // 通过 ne 数组逐渐向后移动
    // 直到遇到空指针
    for(int i = ne[h]; i; i = ne[i])
    {
        cout << e[i] << " ";
    }
    cout << endl << endl;
}
```

时间复杂度：

> 遍历整个链表，时间复杂度为 $O(N)$。

2.1.5 按值查找

（用 ppt 展示一下两种解法）

解法一：遍历整个链表即可。

解法二：如果存储的值数据范围不大，可以用哈希表优化。（不知道哈希表是什么没有关系，只要知道 mp 数组的作用，把它当成一个标记数组即可。）

代码实现：

```cpp
int mp[N]; // mp[i] 表示 i 在这个元素存放的位置

/*
    push_front 和 insert 的时候，打上标记
    mp[x] = id; // x 这个元素存放的位置是 id

    erase 的时候，消除标记
    mp[x] = 0;
*/

// 查询值为 x 的元素存储的位置
int find(int x) // 注意，这里传入的是元素的值
{
    // 策略一：先找到 x，然后返回 x 后面的元素
```

```
    for(int i = ne[h]; i; i = ne[i]) // 遍历链表
    {
        if(e[i] == x) // 如果找到 x
        {
            return i;
        }
    }
    // 找不到就返回 0
    return 0;

    // // 策略二：使用哈希表优化
    return mp[x]; // mp[x] 里面就存着 x 这个元素存储的位置下标
}
```

时间复杂度：

> 遍历整个链表查询：时间复杂度 $O(N)$；
>
> 使用标记数组优化：时间复杂度 $O(1)$。

2.1.6 在任意位置之后插入元素

（ppt 演示插入的流程）

代码实现：

```
// 在存储位置为 p 的元素后面，插入一个元素 x
void insert(int p, int x) // 一定要注意，这里的 p 是位置，不是元素
{
    id++; // x 这个元素分配的位置
    e[id] = x; // 将 x 放在 id 位置处
    ne[id] = ne[p]; // x 指向 p 的后面
    ne[p] = id; // p 指向 x
}
```

时间复杂度：

> 只有常数级别的操作，时间复杂度为 $O(1)$。

2.1.7 删除任意位置之后的元素

（ppt 演示细节，并且展示有可能出现死循环的 bug）

代码实现：

```
// 删除存储位置为 p 后面的元素
void erase(int p) // 注意 p 表示元素的位置
{
    if(ne[p])
    {
        mp[e[ne[p]]] = 0; // 将 p 后面的元素从 mp 中删除
        ne[p] = ne[ne[p]]; // 指向下一个元素的下一个元素
    }
}
```

时间复杂度：

> 只用修改指针，时间复杂度为 $O(1)$。

2.1.8 遗留问题

（ppt 展示一下细节）

为什么不像顺序表一样，实现一个尾插、尾删、删除任意位置的元素等操作?

> 能实现，但是没必要。因为时间复杂度是 $O(N)$ 级别的。
>
> 使用各种数据结构是方便我们去解决问题的，而不是添堵（增加时间复杂度）的~

2.1.9 所有测试代码

```
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 创建
int e[N], ne[N], h, id;
int mp[N]; // mp[i] 表示 i 这个数存储的位置

// 遍历链表
void print()
```

```
13  {
14      for(int i = ne[h]; i; i = ne[i])
15      {
16          cout << e[i] << " ";
17      }
18      cout << endl << endl;
19  }
20
21  // 头插
22  void push_front(int x)
23  {
24      id++;
25      e[id] = x;
26      mp[x] = id; // 标记 x 存储的位置
27
28      // 先让新结点指向头结点的下一个位置
29      // 然后让头结点指向新来的结点
30      ne[id] = ne[h];
31      ne[h] = id;
32  }
33
34  // 按值查找
35  int find(int x)
36  {
37      // // 解法一：遍历链表
38      // for(int i = ne[h]; i; i = ne[i])
39      // {
40      //     if(e[i] == x) return i;
41      // }
42      // return 0;
43
44      // 解法二：用 mp 数组优化
45      return mp[x];
46  }
47
48  // 在任意位置 p 之后，插入一个新的元素 x
49  void insert(int p, int x)
50  {
51      id++;
52      e[id] = x;
53      mp[x] = id;
54
55      ne[id] = ne[p];
56      ne[p] = id;
57  }
58
59  // 删除任意位置 p 后面的元素
```

```
60  void erase(int p)
61  {
62      if(ne[p]) // 当 p 不是最后一个元素的时候
63      {
64          mp[e[ne[p]]] = 0; // 把标记清空
65
66          ne[p] = ne[ne[p]];
67      }
68  }
69
70  int main()
71  {
72      for(int i = 1; i <= 5; i++)
73      {
74          push_front(i);
75          print();
76      }
77
78      //cout << find(1) << endl;
79      //cout << find(5) << endl;
80      //cout << find(6) << endl;
81
82      // insert(1, 10);
83      // print();
84      // insert(2, 100);
85      // print();
86
87      erase(2);
88      print();
89      erase(4);
90      print();
91
92      return 0;
93  }
```

2.2 双向链表的模拟实现

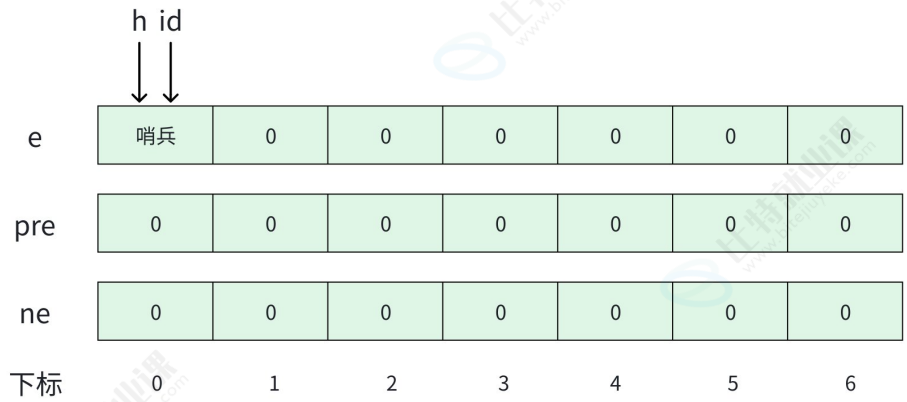
2.2.1 实现方式

（用 ppt 展示细节，效果更佳）

依旧采用静态实现的方式。

双向链表无非就是在单链表的基础上加上一个指向前驱的指针，那就再来一个数组，充当指向前驱的指针域即可。

2.2.2 定义




```
const int N = 1e5 + 10;

int h; // 头结点
int id; // 下一个元素分配的位置
int e[N]; // 数据域
int pre[N], ne[N]; // 前后指针域

// h 默认等于 0，指向的就是哨兵位
// 此时链表为空，没有任何几点，因此 ne[h] = 0
```

2.2.3 头插

（用 ppt 演示细节）

代码实现：

```
// 头插
void push_front(int x)
{
    id++;
    e[id] = x;
    mp[x] = id; // 存一下 x 这个元素的位置

    // 左指向哨兵位，右指向哨兵位的下一个位置，也就是头结点
    pre[id] = h;
    ne[id] = ne[h];

```

```
    // 先修改头结点的指针，再修改哨兵位，顺序不能颠倒
    pre[ne[h]] = id;
    ne[h] = id;
}
```

时间复杂度：

> 只涉及指针的修改，时间复杂度为 $O(1)$。

2.2.4 遍历链表

直接无视 `prev` 数组，与单链表的遍历方式一致。

代码实现：

```
// 打印链表
void print()
{
    for(int i = ne[h]; i; i = ne[i])
    {
        cout << e[i] << " ";
    }
    cout << endl << endl;
}
```

时间复杂度：

> 遍历整个链表，时间复杂度 $O(N)$。

2.2.5 按值查找

> 直接使用 mp 数组优化了。

```
// 查找元素 x 在链表中存储的位置
int find(int x)
{
    // 用 mp 优化
    return mp[x];
```

```
6   }
```

时间复杂度：

> 直接拿值，时间复杂度为 $O(1)$。

2.2.6 在任意位置之后插入元素

（用 ppt 演示流程）

代码实现：

```
1   // 在存储位置为 p 的元素后面，插入一个元素 x
2   void insert_back(int p, int x)
3   {
4       id++;
5       e[id] = x;
6       mp[x] = id; // 存一下 x 这个元素的位置
7
8       // 先左指向 p，右指向 p 的后继
9       pre[id] = p;
10      ne[id] = ne[p];
11
12      // 先让 p 的后继的左指针指向 id
13      // 再让 p 的右指针指向 id
14      pre[ne[p]] = id;
15      ne[p] = id;
16  }
```

时间复杂度：

> 只涉及指针的修改，时间复杂度为 $O(1)$。

2.2.7 在任意位置之前插入元素

（用 ppt 演示流程）

代码实现：

```
1   // 在存储位置为 p 的元素前面，插入一个元素 x
```

```
void insert_front(int p, int x)
{
    id++;
    e[id] = x;
    mp[x] = id; // 存一下 x 这个元素的位置

    // 先左指针指向 p 的前驱，右指针指向 p
    pre[id] = pre[p];
    ne[id] = p;

    // 先让 p 的前驱的右指针指向 id
    // 再让 p 的左指针指向 id
    ne[pre[p]] = id;
    pre[p] = id;
}
```

时间复杂度：

> 只涉及指针的修改，时间复杂度为 $O(1)$。

2.2.8 删除任意位置的元素

（用 ppt 演示流程）

代码实现：

```
// 删除下标为 p 的元素
void erase(int p)
{
    mp[e[p]] = 0; // 从标记中移除

    ne[pre[p]] = ne[p];
    pre[ne[p]] = pre[p];
}
```

时间复杂度：

> 只涉及指针的修改，时间复杂度为 $O(1)$。

2.2.9 所有测试代码

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

// 创建双链表
int e[N], ne[N], pre[N], id, h;
int mp[N]; // mp[i] 表示：i 这个值存储的位置

// 遍历链表
void print()
{
    for(int i = ne[h]; i; i = ne[i])
    {
        cout << e[i] << " ";
    }
    cout << endl << endl;
}

// 头插
void push_front(int x)
{
    id++;
    e[id] = x;
    mp[x] = id;

    // 先修改新来结点的左右指针
    pre[id] = h;
    ne[id] = ne[h];

    // 修改哨兵位下一个结点的左指针
    pre[ne[h]] = id;
    ne[h] = id;
}

int find(int x)
{
    return mp[x];
}

// 在任意位置 p 的后面插入新的元素 x
void insert_back(int p, int x)
{
    id++;
    e[id] = x;
```

```
47      mp[x] = id;
48
49      pre[id] = p;
50      ne[id] = ne[p];
51      pre[ne[p]] = id;
52      ne[p] = id;
53  }
54
55  // 在任意位置 p 的前面插入新的元素 x
56  void insert_front(int p, int x)
57  {
58      id++;
59      e[id] = x;
60      mp[x] = id;
61
62      pre[id] = pre[p];
63      ne[id] = p;
64      ne[pre[p]] = id;
65      pre[p] = id;
66  }
67
68  // 删除任意位置 p 的元素
69  void erase(int p)
70  {
71      mp[e[p]] = 0; // 把标记清空
72
73      ne[pre[p]] = ne[p];
74      pre[ne[p]] = pre[p];
75  }
76
77  int main()
78  {
79      for(int i = 1; i <= 5; i++)
80      {
81          push_front(i);
82          print();
83      }
84
85      //cout << find(3) << endl;
86      //cout << find(5) << endl;
87      //cout << find(0) << endl;
88
89      // insert_front(2, 22);
90      // print();
91      // insert_front(3, 33);
92      // print();
93      // insert_front(4, 44);
```

```
    // print();

    erase(2);
    print();
    erase(4);
    print();

    return 0;
}
```

2.3 循环链表的模拟实现

（可以在 ppt 中展示一下，更清晰）

回看之前实现的带头单向链表。我们定义 0 表示空指针，但其实哨兵位就在 0 位置，所有的结构正好成环。

因此循环链表就是在原有的基础上，让最后一个元素指向表头即可。这里就不带着大家实现了，而且后面会有算法题练习到循环链表~

3. 动态链表 - list（了解）

> new 和 delete 是非常耗时的操作，
>
> 在算法比赛中，一般不会使用 new 和 delete 去模拟实现一个链表。
>
> 而 STL 里面的 list 的底层就是动态实现的双向循环链表，增删会涉及 new 和 delete，效率不高，竞赛中一般不会使用，这里了解一下即可。

3.1 创建 list

```
#include <list>

using namespace std;

int main()
{
    list<int> lt; // 创建一个存储 int 类型的链表

    return 0;
}
```

3.2 push_front / push_back

1. `push_front`：头插；
2. `push_back`：尾插。

时间复杂度：$O(1)$。

测试代码：

```
void testadd()
{
    list<int> lt;
    // 尾插
    for(int i = 1; i <= 5; i++)
    {
        lt.push_back(i);
        print(lt);
    }


    // 头插
    for(int i = 1; i <= 5; i++)
    {
        l.push_front(i);
        print(l);
    }
}
```

3.3 pop_front / pop_back

1. `pop_front`：头删
2. `pop_back`：尾删

时间复杂度：$O(1)$。

测试代码：

```
// 删
void testdelete()
{
    list<int> lt;
```

```
    // 尾插
    for(int i = 1; i <= 5; i++)
    {
        lt.push_back(i);
    }

    // 头插
    for(int i = 5; i >= 1; i--)
    {
        lt.push_front(i);
    }

    // 头删
    for(int i = 1; i <= 3; i++) lt.pop_front();
    // 尾删
    for(int i = 1; i <= 3; i++) lt.pop_back();

    print(lt);
}
```

3.4 所有测试代码

```
#include <iostream>
#include <list>

using namespace std;

void print(list<int>& l)
{
    for(auto x : l)
    {
        cout << x << " ";
    }
    cout << endl;
}

int main()
{
    list<int> l;

    // 尾插
    for(int i = 1; i <= 5; i++)
    {
```

```
        l.push_back(i);
        print(l);
    }

    // 头插
    for(int i = 1; i <= 5; i++)
    {
        l.push_front(i);
        print(l);
    }

    // 头删
    for(int i = 1; i <= 3; i++)
    {
        l.pop_front();
    }
    print(l);

    // 尾删
    for(int i = 1; i <= 3; i++)
    {
        l.pop_back();
    }
    print(l);


    return 0;
}
```

4. 算法题

4.1 排队顺序

> **题目来源：** 洛谷
>
> **题目链接：** B3630 排队顺序
>
> **难度系数：** ★

【题目描述】

> 有 $n(2 \le n \le 10^6)$ 个小朋友，他们的编号分别从 1 到 n。现在他们排成了一个队伍，每个小朋友只知道他后面一位小朋友的编号。现在每个小朋友把他后面是谁告诉你了，同时你还知道排在队首的是哪位小朋友，请你从前到后输出队列中每个小朋友的编号。

【输入描述】

> 第一行一个整数 n ，表示小朋友的人数。
>
> 第二行 n 个整数，其中第 i 个数表示编号为 i 的小朋友后面的人的编号。如果这个数是 0 ，则说明这个小朋友排在最后一个。
>
> 第三行一个整数 h ，表示排在第一个的小朋友的编号。

【输出描述】

> 一行 n 个整数，表示这个队伍从前到后所有小朋友的编号，用空格隔开。

【示例一】

> 输入：
>
> ```
> 6
> 4 6 0 2 3 5
> 1
> ```
>
> 输出：
>
> ```
> 1 4 2 6 5 3
> ```

【解法】

本题相当于告诉了我们每一个点的后继，使用静态链表的存储方式能够很好的还原这个队列。

数组中 $[1,n]$ 的下标可以当做数据域，根据题意修改指针域即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n, h;
int ne[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> ne[i];
    cin >> h;

    // 遍历链表
    for(int i = h; i; i = ne[i])
    {
```

```
19            cout << i << " ";
20        }
21
22        return 0;
23    }
```

4.2 单向链表

题目来源： 洛谷

题目链接： B3631 单向链表

难度系数： ★

【题目描述】

实现一个数据结构，维护一张表（最初只有一个元素 1 ）。需要支持下面的操作，其中 x 和 y 都是 1 到 10^6 范围内的正整数，且保证任何时间表中所有数字均不相同，操作数量不多于 10^5：

- `1 x y` ：将元素 y 插入到 x 后面；
- `2 x` ：询问 x 后面的元素是什么。如果 x 是最后一个元素，则输出 0 ；
- `3 x`：从表中删除元素 x 后面的那个元素，不改变其他元素的先后顺序。

【输入描述】

第一行一个整数 q 表示操作次数。

接下来 q 行，每行表示一次操作，操作具体间题目描述。

【输出描述】

对于每个操作 2 ，输出一个数字，用换行隔开。

【示例一】

输入：

```
6
1 1 99
1 99 50
1 99 75
2 99
3 75
2 1
```

输出：

```
75
99
```

【解法】

链表模板题，直接实现一个单链表即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10, M = 1e6 + 10;

// 链表
int h, id, e[N], ne[N];
int mp[M]; // mp[i] 用来标记 i 这个元素存在什么位置

int main()
{
    int q; cin >> q;

    // 初始化
    id++;
    e[id] = 1;
    mp[1] = id;

    while(q--)
    {
        int op, x, y;
        cin >> op >> x;
        int p = mp[x]; // x 存的位置

        if(op == 1) // x 后面插入
        {
            cin >> y;

            id++;
            e[id] = y;
            ne[id] = ne[p];
            ne[p] = id;

            mp[y] = id; // 标记 y 这个元素存的位置
        }
```

```
37          else if(op == 2) // 查询 x 后面的元素
38          {
39              cout << e[ne[p]] << endl;
40          }
41          else // 删除 x 后面的元素
42          {
43              ne[p] = ne[ne[p]];
44          }
45      }
46
47      return 0;
48  }
```

4.3 队列安排

题目来源： 洛谷

题目链接： P1160 队列安排

难度系数： ★★

【题目描述】

一个学校里老师要将班上 N 个同学排成一列，同学被编号为 $1 \sim N$，他采取如下的方法：

1. 先将 1 号同学安排进队列，这时队列中只有他一个人；
2. $2 \sim N$ 号同学依次入列，编号为 i 的同学入列方式为：老师指定编号为 i 的同学站在编号为 $1 \sim (i-1)$ 中某位同学（即之前已经入列的同学）的左边或右边；
3. 从队列中去掉 M 个同学，其他同学位置顺序不变。

在所有同学按照上述方法队列排列完毕后，老师想知道从左到右所有同学的编号。

【输入描述】

第一行一个整数 N，表示了有 N 个同学。

第 $2 \sim N$ 行，第 i 行包含两个整数 k, p，其中 k 为小于 i 的正整数，p 为 0 或者 1。若 p 为 0，则表示将 i 号同学插入到 k 号同学的左边，p 为 1 则表示插入到右边。

第 $N+1$ 行为一个整数 M，表示去掉的同学数目。

接下来 M 行，每行一个正整数 x，表示将 x 号同学从队列中移去，如果 x 号同学已经不在队列中则忽略这一条指令。

【输出描述】

一行，包含最多 N 个空格隔开的整数，表示了队列从左到右所有同学的编号。

【示例一】

输入：

```
4
1 0
2 1
1 0
2
3
3
```

输出：

```
2 4 1
```

【解法】

频繁的在某一个位置之前和之后插入元素，因此可以用双向循环的链表来模拟。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int h, pre[N], ne[N];
bool st[N]; // st[i] 表示 i 这个同学是否已经出队

int main()
{
    int n; cin >> n;
    // 初始化
    pre[1] = h;
    ne[h] = 1;

    for(int i = 2; i <= n; i++)
    {
        // i
        int k, p; cin >> k >> p;
        if(p == 0) // i 放在 k 的左边
        {
            ne[i] = k;
            pre[i] = pre[k];
```

```
            ne[pre[k]] = i;
            pre[k] = i;
        }
        else // i 放在 k 的右边
        {
            pre[i] = k;
            ne[i] = ne[k];
            pre[ne[k]] = i;
            ne[k] = i;
        }
    }

    int m; cin >> m;
    while(m--)
    {
        int x; cin >> x;
        if(st[x]) continue; // 如果 x 已经被删除，忽视这条指令

        ne[pre[x]] = ne[x];
        pre[ne[x]] = pre[x];
        st[x] = true; // 表示 x 已经被删除
    }

    for(int i = ne[h]; i; i = ne[i])
    {
        cout << i << " ";
    }


    return 0;
}
```

4.4 约瑟夫问题

> **题目来源：** 洛谷
>
> **题目链接：** P1996 约瑟夫问题
>
> **难度系数：** ★

【题目描述】

> n 个人围成一圈，从第一个人开始报数,数到 m 的人出列，再由下一个人重新从 1 开始报数，数到 m 的人再出圈，依次类推，直到所有的人都出圈，请输出依次出圈人的编号。

【输入描述】

> 输入两个整数 n, m ， $1 \leq m, n \leq 100$ 。

【输出描述】

> 输出一行 n 个整数，按顺序输出每个出圈人的编号。

【示例一】

> 输入：
>
> 10 3
>
> 输出：
>
> 3 6 9 2 7 1 8 5 10 4

【解法】

使用循环链表模拟即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110;

int n, m;
int ne[N];

int main()
{
    cin >> n >> m;
    // 创建循环链表
    for(int i = 1; i < n; i++) ne[i] = i + 1;
    ne[n] = 1;

    // 模拟约瑟夫游戏的过程
    int t = n;
    for(int i = 1; i <= n; i++) // 执行 n 次出圈操作
    {
        for(int j = 1; j < m; j++) // 让 t 向后移动 m - 1 次
        {
            t = ne[t];
        }

        cout << ne[t] << " ";
        ne[t] = ne[ne[t]];
```

```
    }

    return 0;
}
```

> 小练习答案：5 -> 4 -> 2 -> 9 -> 8
>
> 希望大家一定要能自己分析出来链表的逻辑结构，理解数据域和指针域所对应的数组里面值的含义。